

Online Games Popularity Prediction

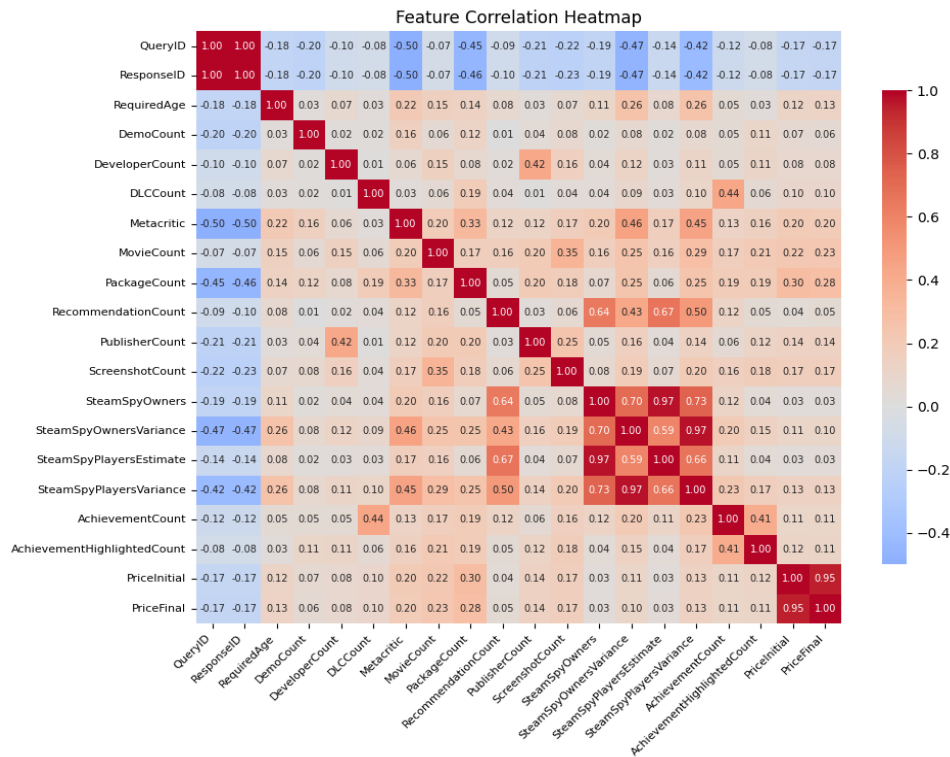
CHP9

Represented by:

Mina Gerges Samaan	20231700252
Mina Maged Naem Boktor	20231700254
Mohamed sherif Badawy	20231700245
Youssef Ashraf Fouad Mahmoud Eldiasty	20231700260
Yousef Ayman Hussein Abdelwahab	20231700261
Merna Nasser Shafik Keroles	20231700251

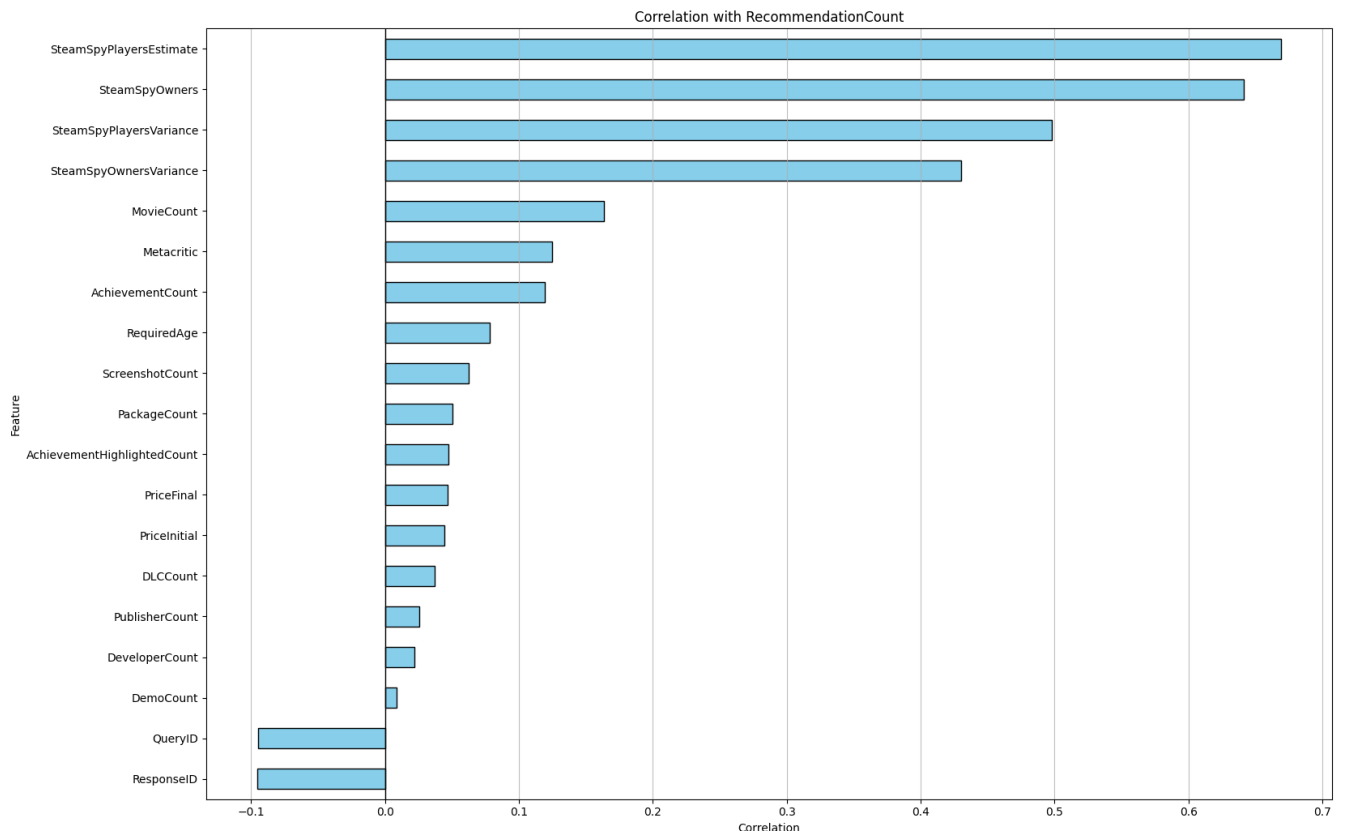
ANALYSIS

1-Correlation between Features



The correlation analysis shows a strong relationship between among several features indicating feature redundancy

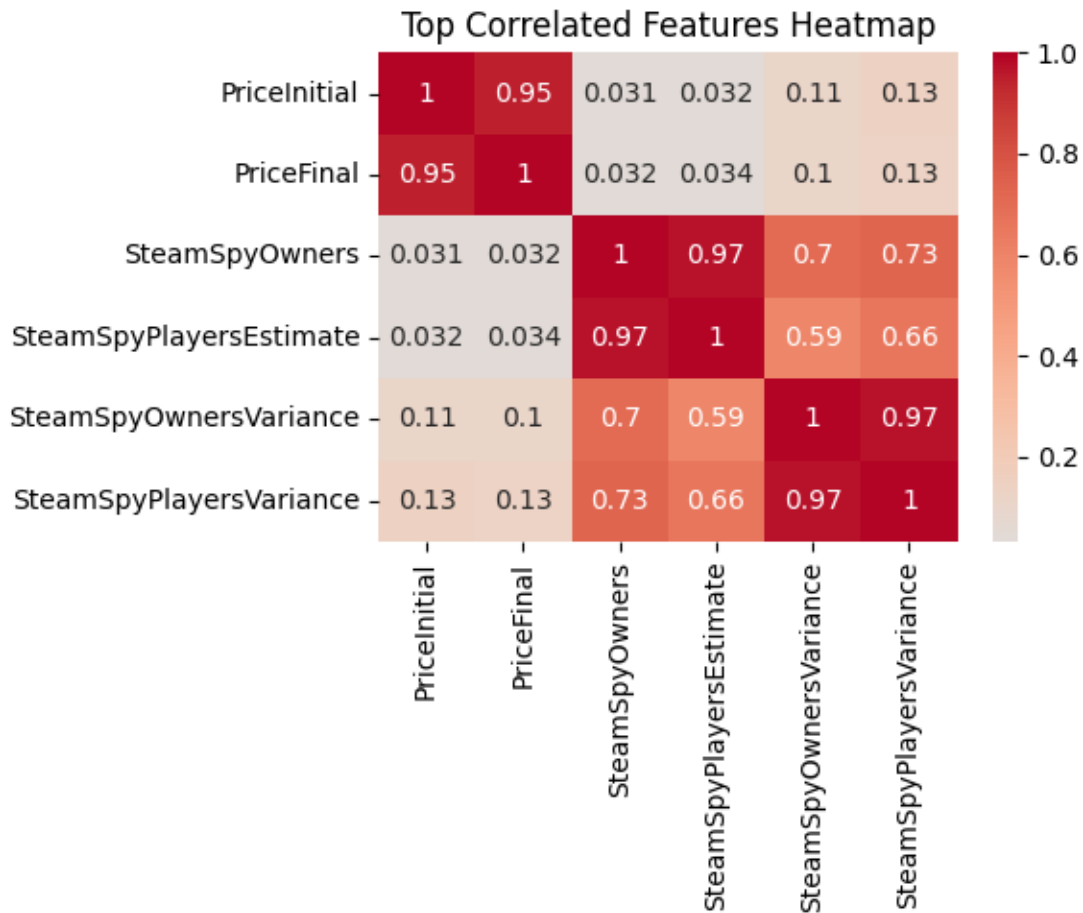
2-Correlation with RecommendationCount



From what we can see that the most Correlated features are

SteamSpyPlayersEstimate	SteamSpyPlayersVariance
SteamSpyOwners	SteamSpyOwnersVariance

3- Top Correlated Features Heatmap



This graph indicates that the correlation between

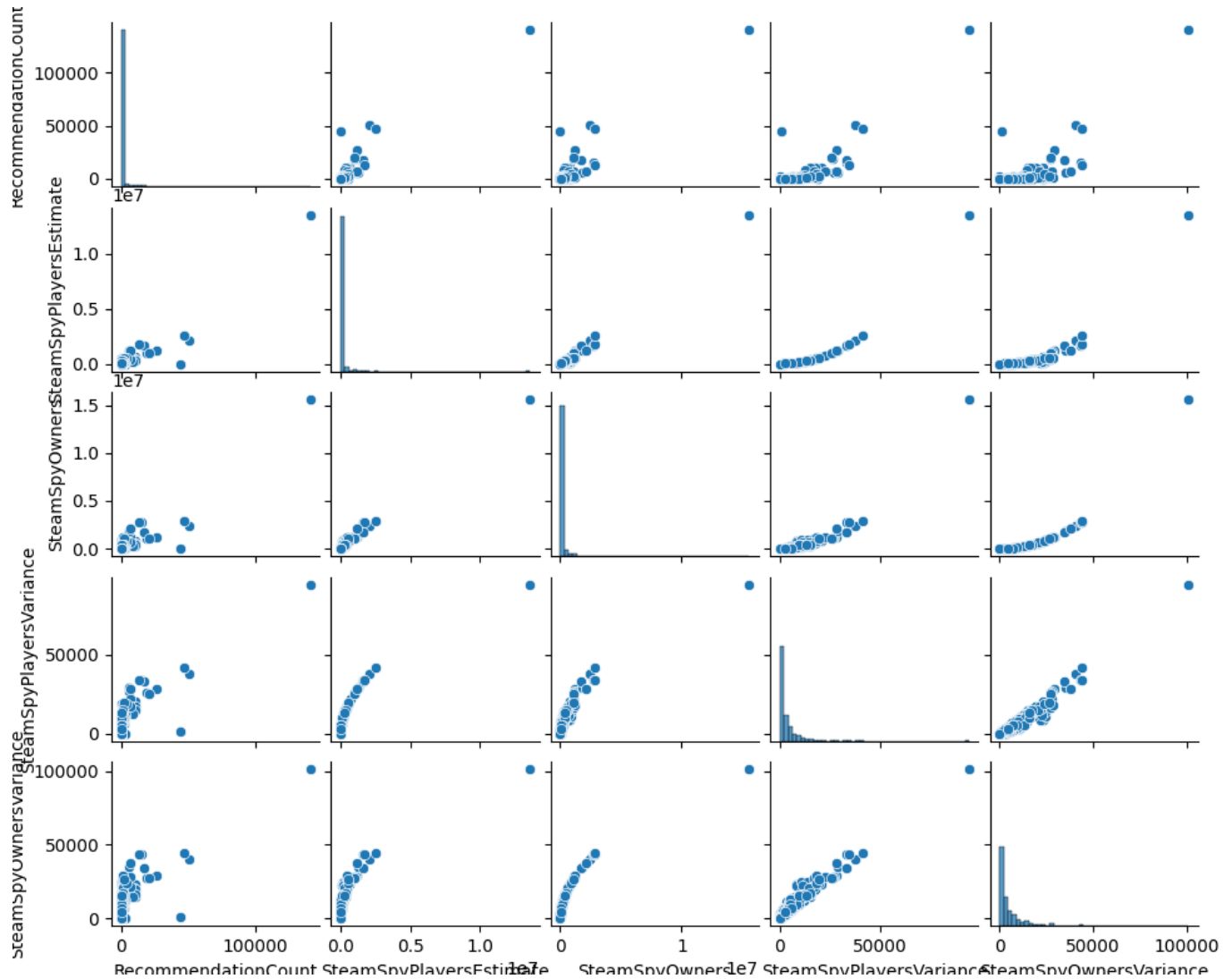
Priceinitial and PriceFinal is (0.95)

SteamSpyOwners and SteamSpyPlayersEstimate is (0.97)

SteamSpyOwnersVariance and SteamSpyPlayersVariance is (0.97)

For each correlated features we must remove one of them to prevent multicollinearity

4- Pair-Wise Relationships



This shows how one variable changes the other variable

Positive: When one goes up the other goes up

Example: More owners → More Players

Negative: When one goes up the other goes down

Preprocessing

1- Data Cleaning:

- Drop Response Identification Columns
[“QueryID”, “ResponseID”, “QueryName”, “ResponseName”]
- Clipping the “PublisherCount” and “DeveloperCount” lower than 1 (illogical)
- Dropping “IsFree” and recreating it due to the inaccuracy in the values with respect to the column “PriceFinal”
- The “PriceCurrency” column contained only USD or NULL values. Therefore, it was converted into a constant binary feature “PriceCurrencyUSD” with a value of 1 for all records, and the original column was dropped after encoding. Drop inaccurate rows where “SteamSpyOwners” = 0 and “RecommendationCount” > 0
- Drop inaccurate rows where “RecommendationCount” = 0 and “SteamSpyOwners” > 0
- Fixing inconsistencies in “SupportedLanguages”
- Imputing missing values using the median of games released in the same year.
- Dropping the rows where SteamSpyOwners > 0 and RecommendationCount = 0 (most likely an error)
- Dropping the rows where SteamSpyOwners = 0 and RecommendationCount > 0 (most likely another error)

2- Encoding:

- Binary Encoding for “RequiredAge” whether it is restricted or not
- Binary Encoding for every column where the values are TRUE and FALSE
- Some columns were binarized to indicate the presence or absence of data, where 1 represents a non-null value and 0 represents a null value
['SupportEmail', 'SupportURL', 'Background', 'HeaderImage', 'Website']
- Binary Encoding for “LegalNotice”

3- Text Preprocessing (NLP):

- Lower case all the strings
- Embedding
['AboutText', 'ShortDescrip', 'DetailedDescrip', 'Reviews']
- Textual features were processed using the pre-trained transformer-based embedding model all-MiniLM-L6-v2, which generates dense vector representations capturing semantic relationships within the text.

4- Feature Extraction:

- Converting raw text like 8 GB RAM, 2 GHZ CPU into numerical features
- Extracting game restrictions from DRMNotice and ExtUserAcctNotice using specific key words

5- Feature Engineering:

- Extracting the “ReleaseYear” and “ReleaseMonth” from “ReleaseDate” and Dividing “ReleaseMonth” into “Month_Sin” and “Month_Cos”
- Extracting how many languages in “SupportedLanguages” into a new column called “LanguageCount”
- Creating new features
 - Game_age $\rightarrow 2026 - \text{ReleaseYear}$
 - New_game $\rightarrow$ if the Game_age < 3
 - Old_game $\rightarrow$ if the Game_age > 10
 - Players_per_Owner $\rightarrow \text{SteamSpyPlayersEstimate} / \text{SteamSpyOwners}$
- Extracting RAM, STORAGE,PROCESSOR GHZ from the function [process_system_requirements] into new columns based on these features
 - ❖ PCMinReqsText
 - ❖ PCRecReqsText
 - ❖ LinuxMinReqsText
 - ❖ LinuxRecReqsText
 - ❖ MacMinReqsText
 - ❖ MacRecReqsText

6- Feature Transformation:

- Power Transformation on all numerical columns using yeo-Johnson method to solve the skewness problem

7-Dimensionality reduction:

- The high-dimensional embeddings were further reduced using Principal Component Analysis (PCA) to solve the (curse of dimensionality) and improve computational efficiency.

FEATURE SELECTION

We removed these features due to their high correlation between them and other features according to our **{ANALYSIS}**

- SteamSpyOwners
- SteamSpyOwnersVariance
- PriceInitial
- ReleaseYear
- date_extraction_failure

MODEL

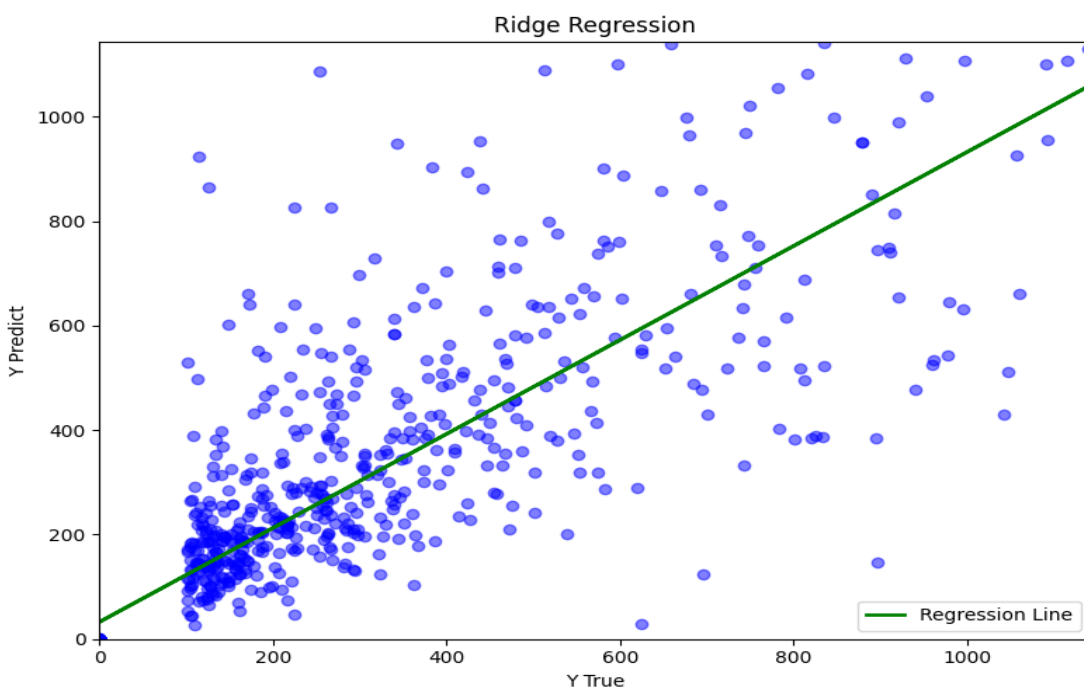
Number of Samples:

- **Training samples: 5136**
- **Testing samples: 1284**
- **Validation strategy: 3-fold (3424 training / 1712 validation per fold)**

We used GridSearchCV to get the best possible hyperparameters for all models, except for Ridge, we used RidgeCV

First: Linear Models (Regularized)

Ridge (Hyperparameters alpha: 0.01)



Resulted Metrics

MAE: 534.1445

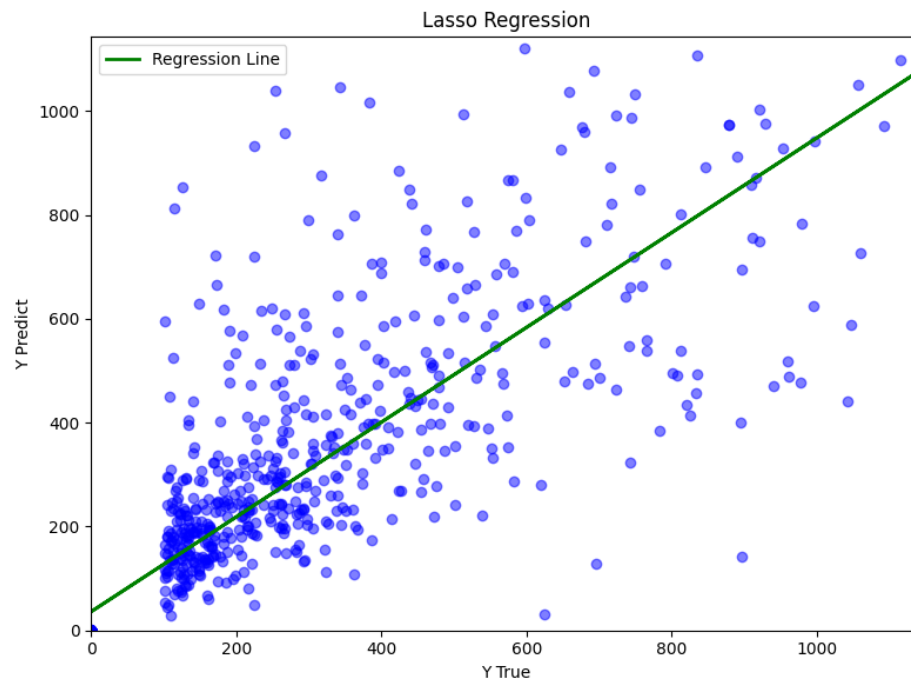
MSE: 5092186.7803

RMSE: 2256.5874

R2: 0.9683

The good performance of the Ridge Regressor over more complex non-linear ensembles, such as Gradient Boosting and Random Forest, indicates that the underlying relationship between the features and the target variable (RecommendationCount) is mostly linear.

Lasso (Hyperparameters alpha: 0.01)



Resulted Metrics

MAE: 660.6385

MSE: 12766373.0636

RMSE: 3573.0062

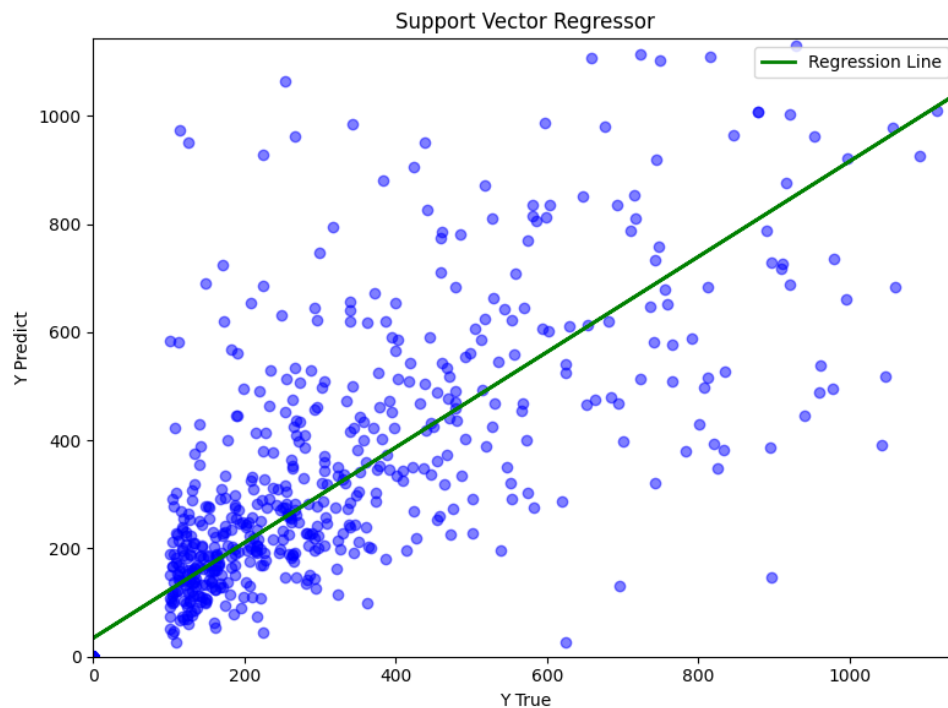
R2: 0.9206

Lasso uses L1 regularization, which can shrink some coefficients exactly to zero, effectively performing feature selection by keeping only the most important variables. However, in our case, Ridge achieved better performance than Lasso, suggesting that eliminating features led to a loss of useful information.

Second: Kernal Model

Support Vector Regressor

(Hyperparameters C: 10.0, kernel: linear)



Resulted Metrices

MAE: 561.9609

MSE: 5909749.3049

RMSE: 2430.9976

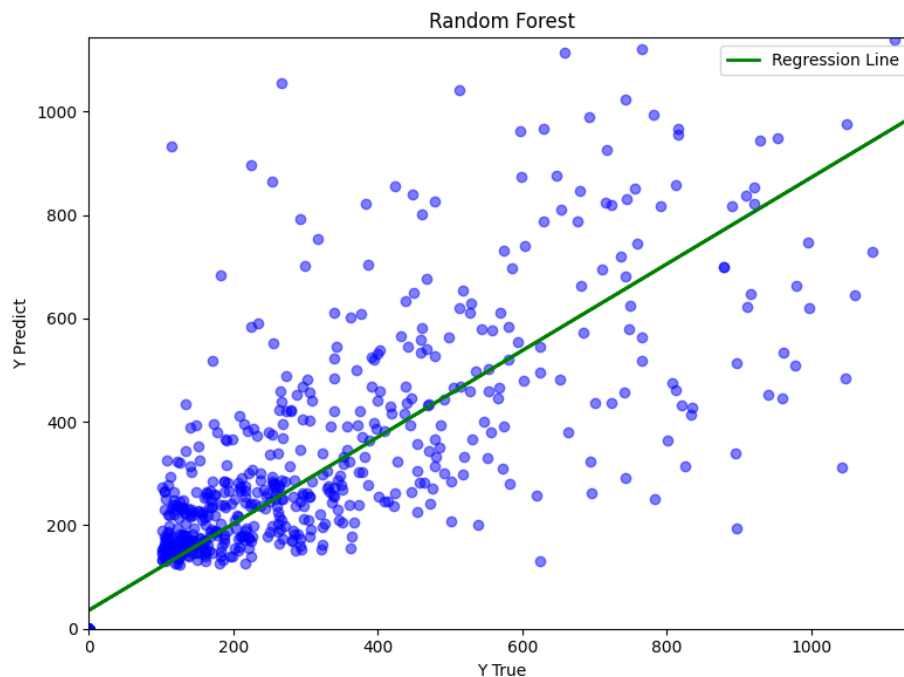
R2: 0.9632

Since the linear kernel performed best, this enforces even further that the data has linear structure

Third: Tree Models

Random Forest (Hyperparameters)

(max_depth: None, min_samples_leaf: 1, n_estimators: 200)



Resulted Metrics

MAE: 582.3882

MSE: 9061782.7920

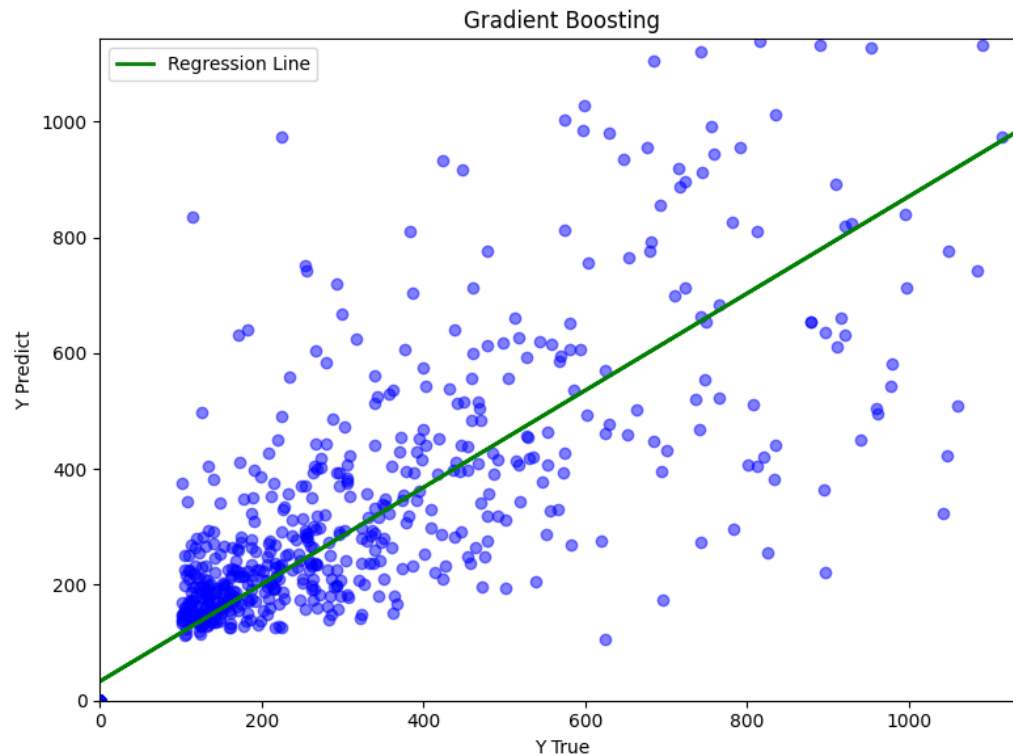
RMSE: 3010.2795

R2: 0.9436

The fact that Ridge performed better than Random Forest suggests that the complexity of the trees might be capturing noise rather than actual patterns, even with 200 estimators.

Gradient Boosting (Hyperparameters)

(learning_rate: 0.05, max_depth: 5, n_estimators: 200)



Resulted Metrics

MAE: 595.0055

MSE: 10561229.3927

RMSE: 3249.8045

R2: 0.9343

The higher RMSE suggests that the model may be capturing noise or is sensitive to hyperparameter choices.

CONCLUSION

At the start of this project, we thought that the number of players would have a big impact on how popular a game was RecommendationCount

Ultimately, our models reinforce this idea by demonstrating that player-related features have the strongest predictive signal, even when combined with a broader range of variables.

We detected highly correlated features causing multicollinearity. While Ridge regression remains robust in terms of prediction due to L2 regularization, coefficient allocation among correlated variables becomes unstable and sensitive to changes in correlation structure, which can affect interpretability and feature importance consistency under distribution shifts.

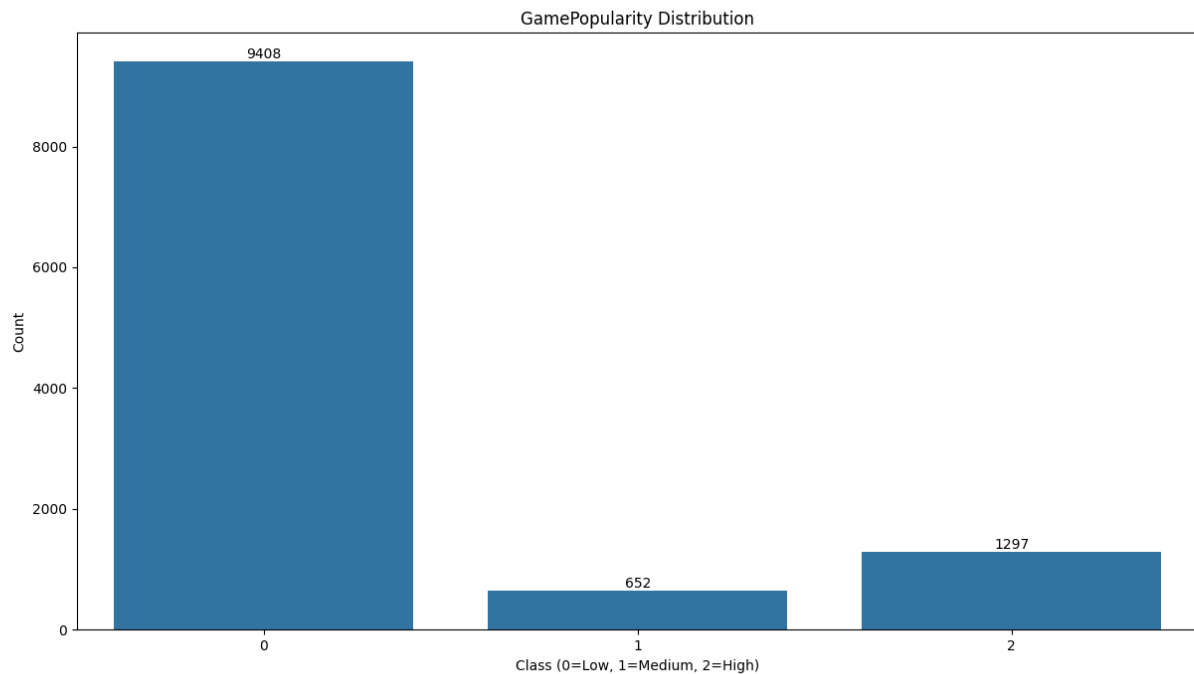
We were able to create a highly predictive model for game recommendations, and Ridge Regression achieved an R^2 score of **0.9683**, which is very good. The strong performance of a linear kernel in the Support Vector Regressor also shows that the relationship between the features and RecommendationCount is mostly linear.

We improved the feature space and the model's performance by using preprocessing steps like power transformation, extracting structured numerical values from raw text, and adding NLP embeddings for game descriptions. In general, the results support the idea that while many factors affect how accurate predictions are, the number of players is still the most important factor in making a game popular.

Online Games Popularity Classification

ANALYSIS

1-Class imbalance



From this graph, it's clear that the dataset isn't balanced, this shows that the Low class dominates the dataset, which could make the model focus more on predicting Low and perform worse on the other classes.

Preprocessing (Classification)

1 - Resampling (Oversampling):

- To solve this problem, I used SMOTE to increase the number of samples in the minority classes and make the dataset more balanced. I also applied class weights during training, so the model gives more importance to the Medium and High classes instead of focusing on the Low class
-

2 - Encoding (Label Encoding):

- The target y into {Low: 0, Medium :1, High :2}
-

3 - Feature Engineering:

- Creating new features
- 'Content_to_Price_Ratio' = 'ContentTotal' / 'PriceFinal' + 0.01
- 'Owner_Base_Pace' = 'SteamSpyOwners' / ('Game_age' + 1)
- 'Hardware_Tax' = 'PCMinReqsText_Processor_GHz' / ('PriceFinal'+ 0.01)
- 'Dev_to_Pub_Ratio' = 'DeveloperCount' / 'PublisherCount' + 0.01)
- 'Is_Discounted' = 'PriceInitial' > 'PriceFinal'

- 'Discount_Amount' = 'PriceInitial' - 'PriceFinal'
- 'Platform_Count' = 'PlatformWindows' + 'PlatformLinux' + 'PlatformMac'
- 'Discount_Percentage' = 'Discount_Amount' / ('PriceInitial' + 0.01)

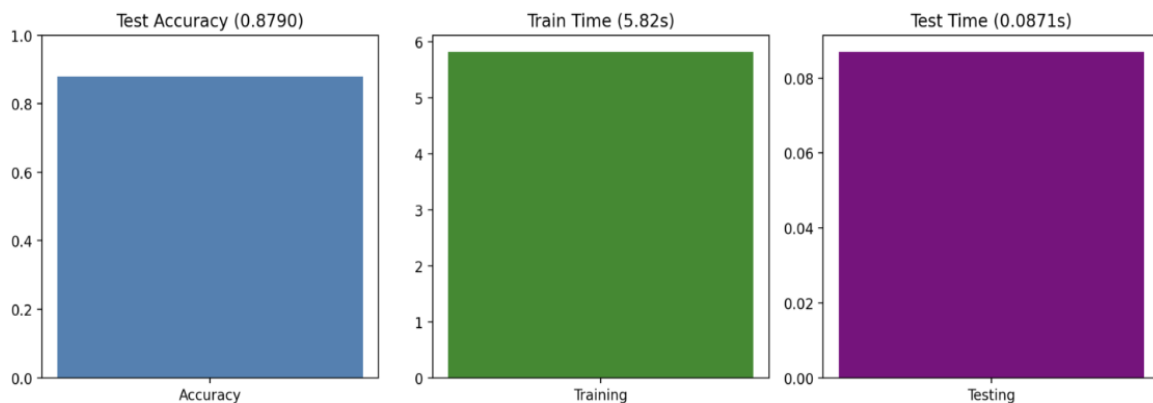
FEATURE SELECTION

We removed these features due to their high correlation between them and other features according to our **{ANALYSIS}**

- SteamSpyOwners
- SteamSpyOwnersVariance
- PriceInitial
- ReleaseYear
- date_extraction_failure

HYPERPARAMETER TUNING

Random Forest



Hyperparameters:

n_estimators:

100 -> 87.50%

300 -> 87.90%

500 -> 88.07%

max_depth:

8 -> 85.65%

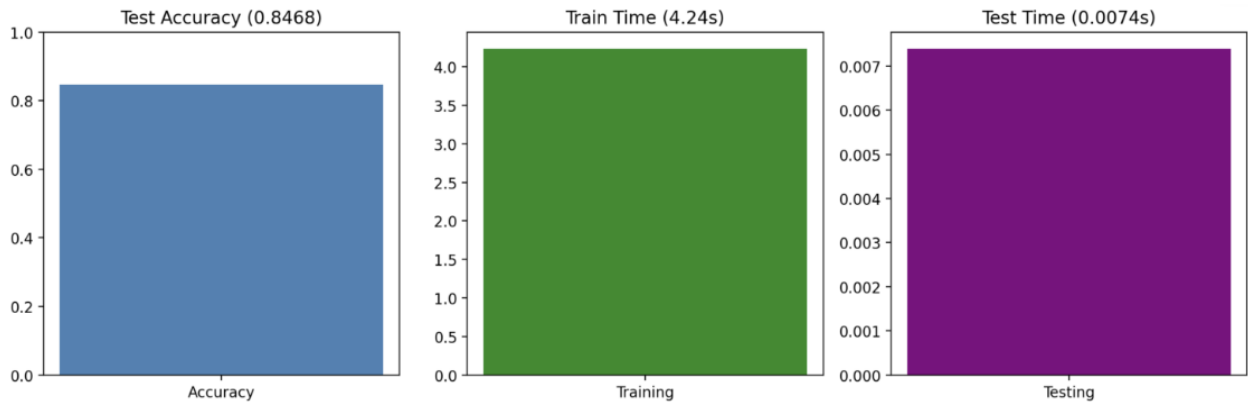
12 -> 87.90%

20 -> 85.48%

n_estimators: Increasing trees improves performance due to better averaging.

Max_depth: Optimal around 12 to avoid overfitting or underfitting

Decision Tree



Hyperparameters:

min_samples_split:

2 -> 84.68%

10 -> 84.68%

100 -> 84.60%

max_depth:

3 -> 81.56%

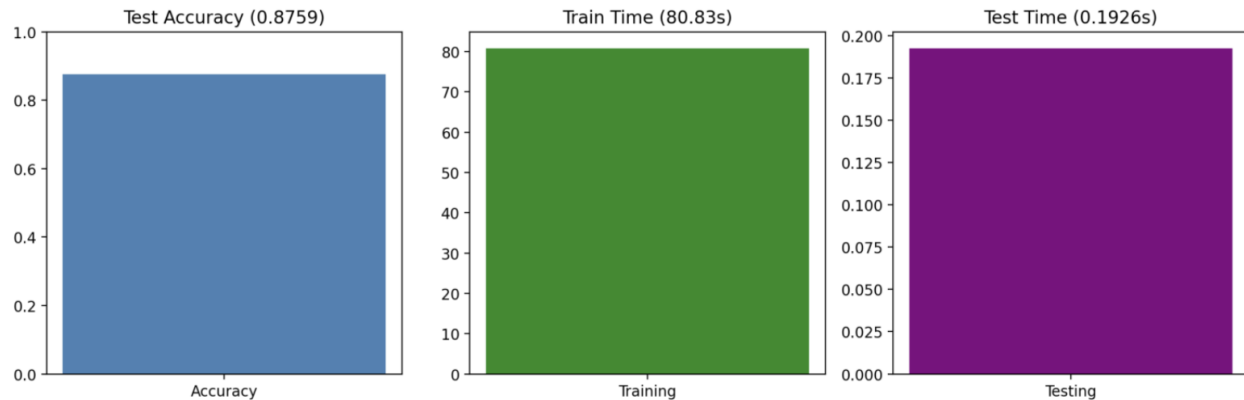
5 -> 84.68%

60 -> 86.40%

min_samples_split: After a specific number, the model starts to overfit.

Max_depth: The model doesn't tend to overfit widely on high depths, but suffers from severe underfitting when decreased.

AdaBoost Classifier



Hyperparameters:

n_estimators:

50 -> 83.67%

200 -> 87.59%

400 -> 89.00%

learning_rate:

0.1 -> 83.76%

1.0 -> 87.59%

1.5 -> 86.00%

n_estimators: Increasing estimators improves performance but gains become smaller after a specific number of estimators.

LR (eta): Using high learning doesn't allow the model to converge while low learning rate make the model learning very slow.

CONCLUSION

The primary challenge in this dataset was the heavy class imbalance between the 'medium' (652 samples) and 'low' (9,408 samples) classes. To address this, SMOTE and class weights were applied, which slightly improved overall model accuracy. However, these techniques were not entirely sufficient for the AdaBoost model, which ultimately correctly predicted the 'medium' class only 39% of the time. This highlights the ongoing difficulty of minority class prediction in highly skewed datasets.